Pontificia Universidad Javeriana

Ricardo Hurtado Forero, Jose Manuel Guerrero López, Samuel Enrique Sabogal Giraldo

Jhon Corredor

Sistemas Distribuidos

10 de Abril 2026

1. Introducción

El presente informe describe el diseño e implementación inicial de una plataforma distribuida para la gestión inteligente del tráfico urbano. El sistema tiene como objetivo monitorear, analizar y reaccionar ante condiciones de tráfico en tiempo real, utilizando múltiples componentes distribuidos que se comunican a través de la biblioteca ZeroMQ.

La plataforma simula una ciudad representada como una cuadrícula de 3x3 intersecciones (filas A–C, columnas 1–3), donde cada intersección cuenta con sensores de tráfico de tres tipos: cámaras de tráfico, espiras inductivas y sensores GPS simulados. El sistema distribuye sus componentes en tres máquinas físicas o virtuales (PC1, PC2 y PC3), cada una con responsabilidades claramente definidas.

Este informe incluye los modelos del sistema, el diseño completo con diagramas UML, las reglas de control de tráfico, el protocolo de pruebas y la estrategia de obtención de métricas de rendimiento, así como la descripción de la implementación inicial de los servicios en PC1 y PC2.

2. Modelos del Sistema

2.1 Modelo Arquitectónico

El sistema sigue una arquitectura distribuida de capas, donde cada capa tiene una responsabilidad bien definida y se comunica con las demás mediante patrones de mensajería asíncrona y síncrona proporcionados por ZeroMQ.

Capa	Máquina	Responsabilidad
Sensores y Broker	PC1	Generación de eventos de tráfico y enrutamiento mediante patrón PUB/SUB
Analítica y Control	PC2	Procesamiento de eventos, detección de congestión, control de semáforos y réplica de BD
Monitoreo y Persistencia	PC3	Base de datos principal, consultas históricas, monitoreo y comandos manuales del usuario

La comunicación entre capas utiliza tres patrones ZeroMQ: PUB/SUB para la transmisión de eventos desde los sensores hasta el servicio de analítica, PUSH/PULL para la escritura asíncrona en las bases de datos, y REQ/REP para las consultas síncronas del módulo de monitoreo.

2.2 Modelo de Interacción

El flujo de interacción del sistema sigue la siguiente secuencia operacional:

1. Los sensores (cámaras, espiras, GPS) en PC1 generan eventos periódicos con datos de tráfico de las 9 intersecciones de la cuadrícula 3x3.

2. Los eventos se publican al Broker ZeroMQ en PC1 mediante el patrón PUB/SUB, organizados por tópico: 'camara', 'espira' y 'gps'.
3. El Broker reenvía los eventos al Servicio de Analítica en PC2, también mediante PUB/SUB.
4. El Servicio de Analítica evalúa los eventos contra las reglas de congestión y toma decisiones de control.
5. Si se detecta un cambio de estado, el Servicio de Analítica envía comandos al Servicio de Control de Semáforos (PUSH/PULL) y actualiza las bases de datos principal (PC3) y réplica (PC2) de forma asíncrona.
6. El Módulo de Monitoreo en PC3 permite al usuario consultar el estado actual e histórico, y enviar comandos directos (ej. prioridad ambulancia) al Servicio de Analítica mediante REQ/REP.

Toda la comunicación entre componentes es no bloqueante, excepto las consultas REQ/REP del módulo de monitoreo, que son síncronas por naturaleza.

2.3 Modelo de Fallos

El principal fallo contemplado es la caída del PC3 (nodo de persistencia principal y monitoreo). El sistema maneja este escenario de la siguiente forma:

- Detección automática: El Servicio de Analítica implementa un mecanismo de Health Check periódico hacia PC3. Si no recibe respuesta en 3 intentos consecutivos (timeout de 2 segundos cada uno), declara a PC3 como inactivo.
- Conmutación transparente (Failover): Ante la detección del fallo, todos los componentes redirigen automáticamente sus escrituras hacia la Base de Datos Réplica ubicada en PC2. Esta operación es transparente para el usuario.
- Recuperación: Cuando PC3 vuelve a estar disponible, el sistema detecta su recuperación mediante el Health Check y puede sincronizar los datos faltantes desde la réplica.

Adicionalmente, se contempla la pérdida de mensajes en la red. ZeroMQ maneja internamente colas de mensajes y reconexión automática entre sockets, lo que mitiga la pérdida de eventos por fallos de red transitorios.

2.4 Modelo de Seguridad

Para la primera versión del sistema se contemplan los siguientes aspectos de seguridad básicos:

- Validación de mensajes: Cada mensaje recibido por el Servicio de Analítica es validado contra el esquema JSON esperado. Mensajes malformados son descartados y registrados en log.
- Control de acceso: El Módulo de Monitoreo (PC3) es el único punto de entrada para comandos manuales. No existe acceso directo externo al Servicio de Analítica ni a las bases de datos.
- Integridad de datos: Las escrituras en la base de datos incluyen timestamp y sensor_id para permitir trazabilidad y detección de duplicados.

3. Parámetros de Configuración del Sistema

3.A Definición Inicial de Recursos

Los procesos obtienen su configuración inicial desde un archivo de configuración compartido (config.json) que es leído al inicio de cada proceso. Este archivo define todos los parámetros del sistema de forma centralizada, facilitando su modificación sin necesidad de alterar el código fuente.

Cuadrícula de la ciudad

- Tamaño de la matriz: 3 filas (A, B, C) × 3 columnas (1, 2, 3) = 9 intersecciones.
- Intersecciones: INT_A1, INT_A2, INT_A3, INT_B1, INT_B2, INT_B3, INT_C1, INT_C2, INT_C3.
- Cada intersección tiene un semáforo y es cubierta por los 3 tipos de sensores.

Sensores

- 1 proceso de sensor tipo Cámara → cubre las 9 intersecciones (genera eventos con intersección aleatoria de la cuadrícula).
- 1 proceso de sensor tipo Espira Inductiva → cubre las 9 intersecciones.
- 1 proceso de sensor tipo GPS → cubre las 9 intersecciones.
- Intervalo de generación de eventos: 5 segundos por sensor (configurable en config.json).

Semáforos

- 9 semáforos, uno por intersección. Estado inicial: todos en VERDE.
- Estados posibles: VERDE (duración base: 15 s) y ROJO (duración base: 15 s).
- No se usa luz amarilla.

3.B Reglas de Control de Tráfico

El sistema define tres estados de circulación con sus respectivas reglas y acciones. Las variables utilizadas son: Lq (longitud de cola, vehículos), Vp (velocidad promedio, km/h) y Cv (conteo vehicular, vehículos en 30 seg).

Estado	Regla de detección	Acción semáforo	Tiempo verde/rojo
Tráfico Normal	Lq < 5 AND Vp > 35 AND Cv < 10	Ciclo estándar	Verde: 15 s / Rojo: 15 s
Congestión	Lq ≥ 5 OR Vp ≤ 20 OR Cv ≥ 20	Ampliar verde en vía congestionada	Verde: 30 s / Rojo: 15 s

Priorización (Ambulancia)	Comando manual del usuario desde PC3	Verde inmediato en vía indicada, rojo en el resto	Verde: 60 s (fijo)

Las reglas son evaluadas por el Servicio de Analítica cada vez que recibe un evento. Si múltiples intersecciones presentan congestión simultánea, se priorizan en orden de severidad (mayor Lq primero).

3.C Tipos de Consulta del Módulo de Monitoreo

El usuario puede realizar los siguientes tipos de consulta y comandos desde PC3:

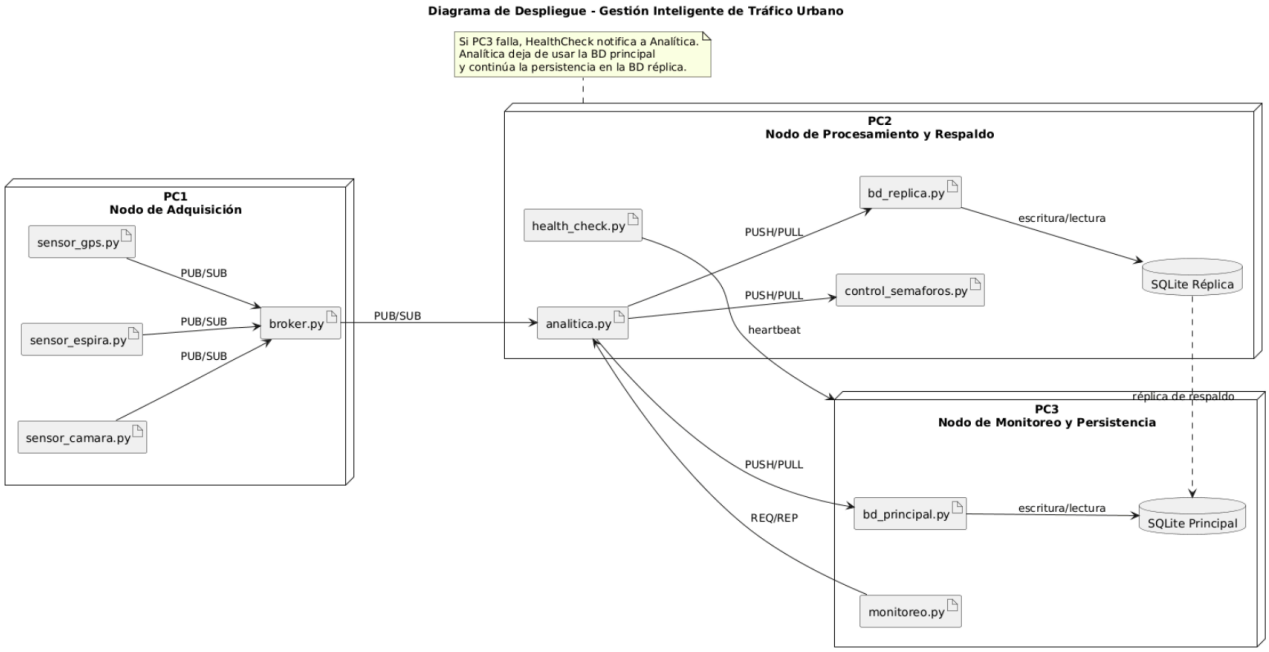
- CONSULTA_ESTADO_ACTUAL: Retorna el estado actual de todos los semáforos y el nivel de congestión de cada intersección.
- CONSULTA_HISTORICO [interseccion] [hora_inicio] [hora_fin]: Retorna todos los eventos registrados para una intersección en un rango de tiempo. Ejemplo: CONSULTA_HISTORICO INT_B2 2026-04-10T08:00:00Z 2026-04-10T09:00:00Z
- CONSULTA_CONGESTION_HISTORICA: Retorna todas las detecciones de congestión registradas en la base de datos.
- PRIORIZAR_VIA [interseccion]: Fuerza el verde inmediato en la intersección indicada. Ejemplo: PRIORIZAR_VIA INT_A1
- CAMBIO_MANUAL [interseccion] [estado]: Cambia el estado de un semáforo manualmente. Estado puede ser VERDE o ROJO. Ejemplo: CAMBIO_MANUAL INT_C3 ROJO

Ejemplo de indicación directa: el operador observa en el sistema de monitoreo que una ambulancia requiere paso prioritario por la vía A. Emite el comando PRIORIZAR_VIA INT_A2, el módulo de monitoreo envía la solicitud al Servicio de Analítica mediante REQ/REP, y este ordena al Servicio de Control de Semáforos el cambio inmediato a verde en INT_A2 por 60 segundos.

4. Diseño del Sistema

4.1 Diagrama de Despliegue

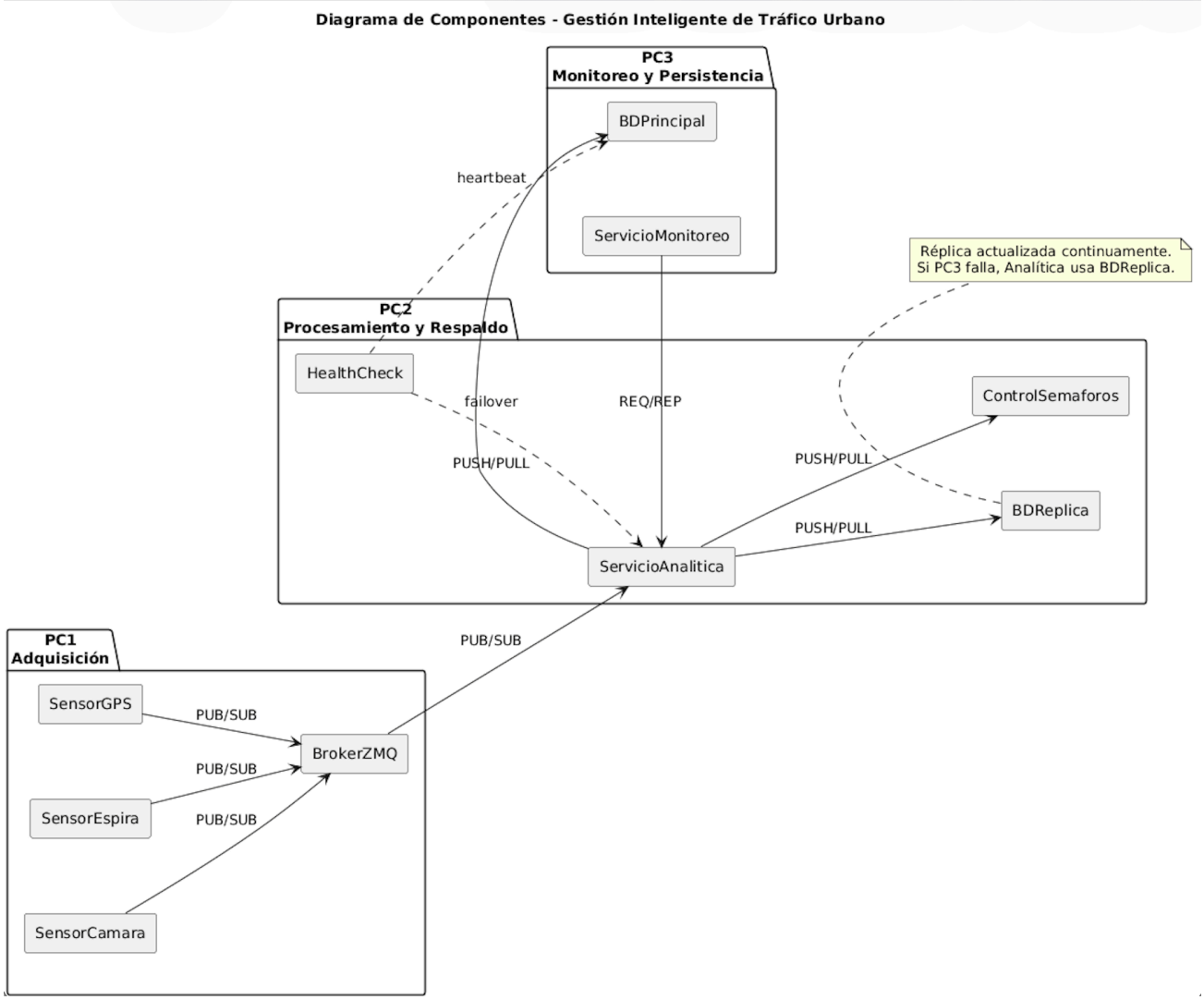
El diagrama de despliegue presenta la distribución física del sistema en tres nodos. PC1 concentra la generación e ingesta de eventos de tráfico mediante sensores simulados y un broker ZeroMQ. PC2 ejecuta el procesamiento principal del sistema, incluyendo la analítica, el control de semáforos, la base de datos réplica y el mecanismo de detección de fallos. PC3 aloja el módulo de monitoreo y la base de datos principal. Esta distribución favorece el desacoplamiento funcional, la persistencia distribuida y la continuidad operativa ante la caída del nodo principal de almacenamiento.



[Figura 1 — Diagrama de Despliegue adjunto en los artefactos de diseño]

4.2 Diagrama de Componentes

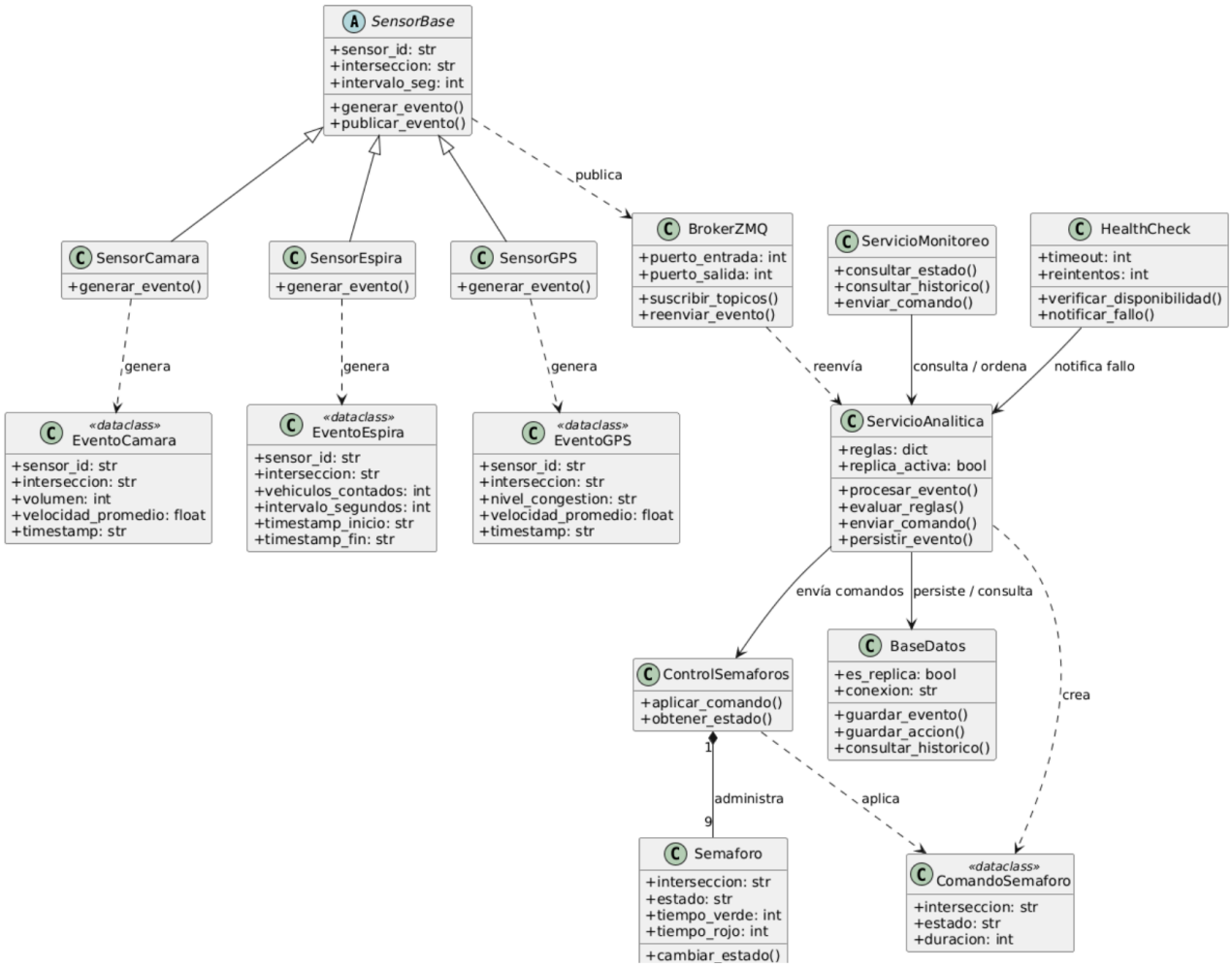
El diagrama de componentes muestra la organización lógica del sistema distribuido. En PC1 se ubican los componentes productores de eventos y el broker encargado del desacoplamiento de la mensajería. En PC2 reside el núcleo de procesamiento, donde el servicio de analítica interpreta los eventos, toma decisiones y coordina tanto el control de semáforos como la persistencia. En PC3 se localizan el monitoreo y la base de datos principal. La presencia de una réplica en PC2 y de un componente de detección de fallos permite mantener la operación del sistema ante la caída de PC3.



[Figura 2 — Diagrama de componentes]

4.3 Diagrama de Clases

El diagrama de clases presenta la estructura lógica principal del sistema y su posible traducción a una implementación orientada a objetos en Python. La jerarquía parte de una clase abstracta SensorBase, de la cual heredan los sensores de cámara, espira y GPS, cada uno responsable de generar eventos específicos del tráfico. El BrokerZMQ desacopla la publicación y recepción de eventos, mientras que ServicioAnalitica centraliza el procesamiento, la evaluación de reglas, la persistencia y la generación de comandos de control. El subsistema de semaforización está compuesto por ControlSemaforos y múltiples objetos Semaforo, y la persistencia se abstrae mediante la clase BaseDatos, reutilizable para la base principal y la réplica. Finalmente, ServicioMonitoreo y HealthCheck representan, respectivamente, la interacción con el usuario y la detección de fallos dentro del sistema distribuido.



[Ver Figura 3 — Diagrama de Clases]

4.4 Diagrama de Secuencia

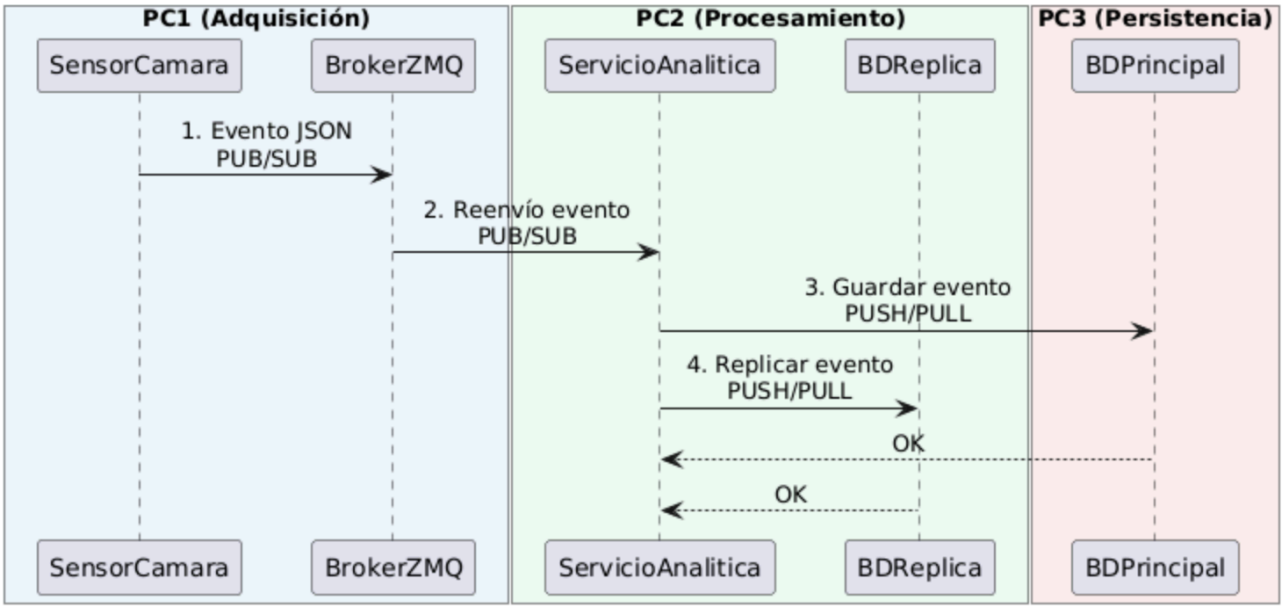
El diagrama de secuencia presentado ilustra el flujo normal de procesamiento de un evento dentro del sistema distribuido de gestión de tráfico urbano. El proceso inicia cuando el sensor de cámara genera un evento en formato JSON, el cual es publicado mediante el patrón PUB/SUB hacia el broker de mensajería.

El broker actúa como un componente de desacoplamiento, reenviando el evento al servicio de analítica sin que exista una dependencia directa entre productores y consumidores. Posteriormente, el servicio de analítica recibe, valida y procesa la información, constituyéndose como el núcleo de toma de decisiones del sistema.

Una vez procesado el evento, este es persistido tanto en la base de datos principal como en la base de datos réplica mediante el patrón PUSH/PULL, garantizando la disponibilidad y redundancia

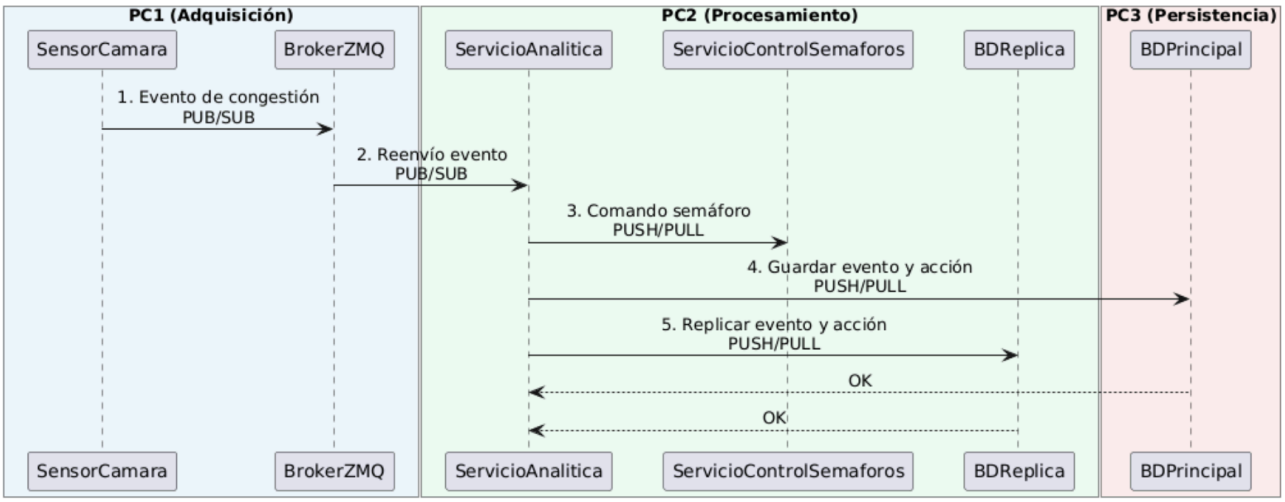
de la información. Finalmente, ambas bases de datos confirman el almacenamiento exitoso del evento.

Este flujo refleja un diseño desacoplado, escalable y tolerante a fallos, donde la separación de responsabilidades y el uso de patrones de comunicación asíncrona permiten mantener la continuidad operativa del sistema.



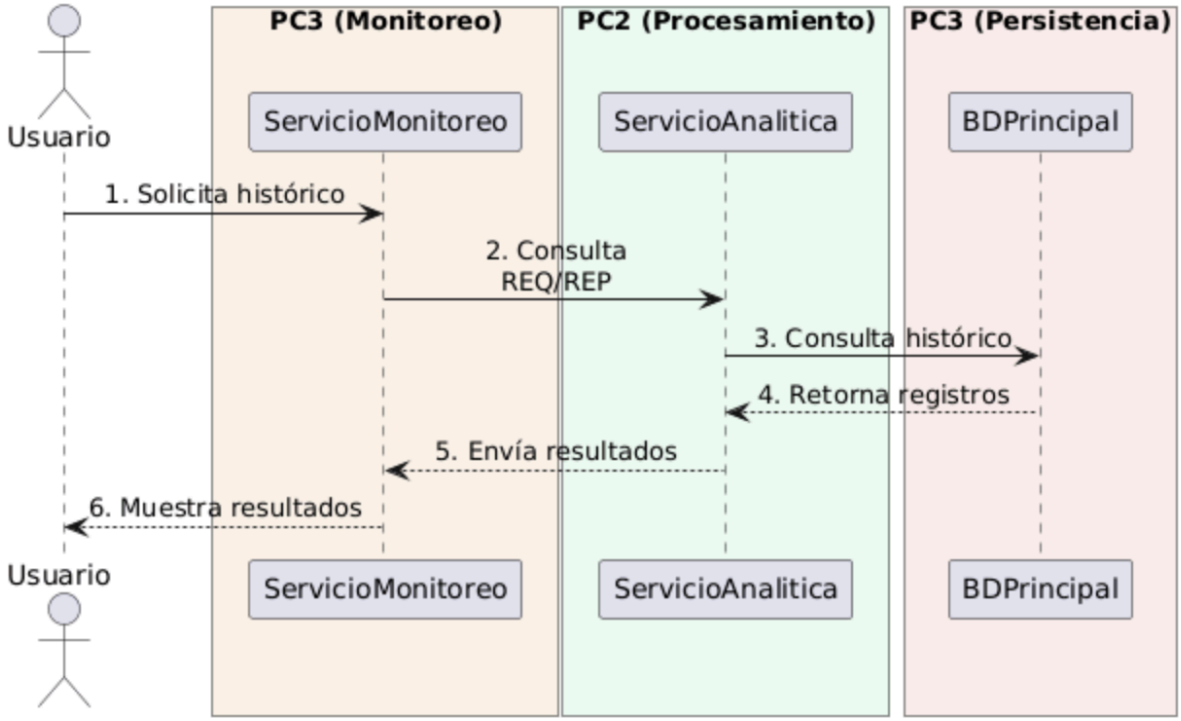
[Figura 3 — Diagrama de flujo normal del evento]

El diagrama de secuencia presenta el caso de detección de congestión y la respuesta del sistema sobre la semaforización. El flujo inicia cuando un sensor de cámara genera un evento asociado a una condición de congestión y lo publica al broker mediante PUB/SUB. El broker reenvía el evento al servicio de analítica, donde se aplican las reglas definidas para identificar la situación de tráfico. Una vez detectada la congestión, la analítica envía un comando al servicio de control de semáforos para modificar el estado de la intersección afectada. Finalmente, tanto el evento recibido como la acción ejecutada se almacenan en la base de datos principal y en la réplica, asegurando persistencia y respaldo de la información.



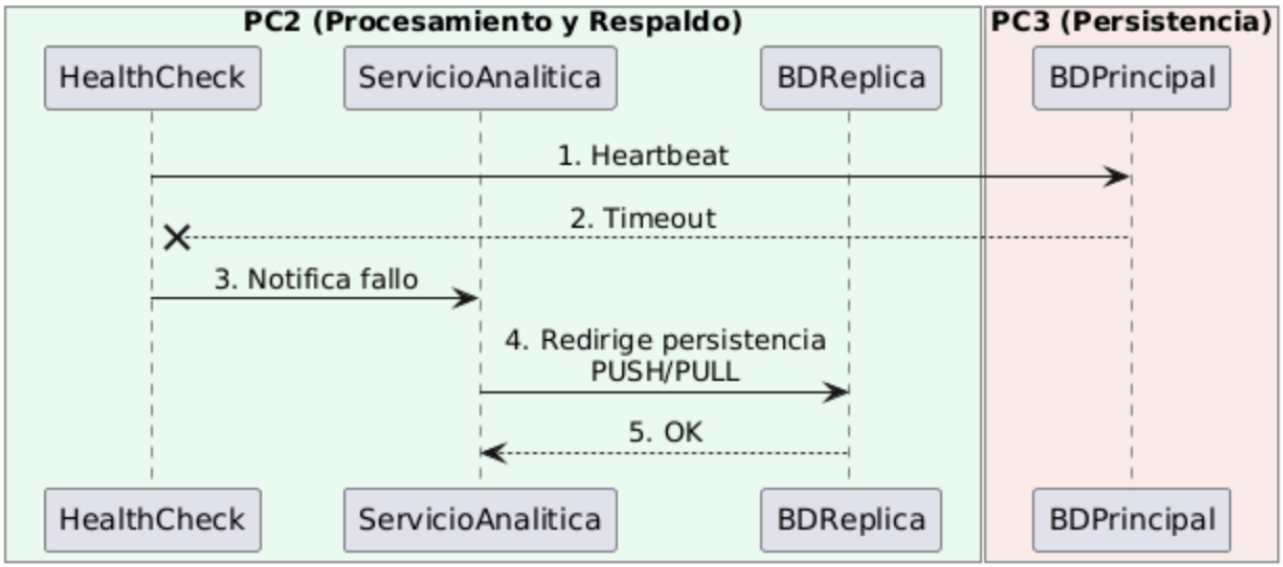
[Figura 4 — Diagrama de secuencia de detección de congestión y cambio de semáforo]

El diagrama de secuencia muestra el flujo de una consulta histórica realizada por el usuario. La solicitud se recibe en el servicio de monitoreo, que actúa como punto de entrada del sistema para consultas y comandos manuales. Posteriormente, el servicio de monitoreo envía la petición al servicio de analítica mediante el patrón REQ/REP. El servicio de analítica consulta la base de datos principal, recupera los registros correspondientes al rango solicitado y devuelve la respuesta al módulo de monitoreo, el cual presenta finalmente los resultados al usuario. Este flujo evidencia la separación entre la capa de interacción, la lógica del sistema y la persistencia.



[Figura 5 — Diagrama de secuencia consulta histórica del usuario]

El diagrama de secuencia representa el escenario de falla del nodo PC3 y la activación del mecanismo de tolerancia a fallos del sistema. El proceso inicia cuando el componente HealthCheck, ubicado en PC2, realiza verificaciones periódicas de disponibilidad sobre la base de datos principal en PC3. Ante la ausencia de respuesta, el sistema interpreta la situación como una falla del nodo principal. Posteriormente, HealthCheck notifica al servicio de analítica, el cual redirige la persistencia hacia la base de datos réplica ubicada en PC2. De esta manera, el sistema continúa almacenando información sin interrumpir el procesamiento de eventos, cumpliendo el requisito de continuidad operativa frente a la caída de PC3.



[Figura 6 —Diagrama de secuencia (caída de PC3 y failover hacia la BD réplica)]

5. Protocolo de Pruebas

5.1 Tipos de Pruebas

El protocolo de pruebas cubre todos los tipos relevantes para un sistema distribuido:

Tipo de Prueba	Descripción y Casos
Funcionales	Verificar que cada componente cumple su función: generación de eventos, detección de congestión, cambio de semáforos, consultas y comandos manuales.
Integración	Verificar la comunicación entre PC1↔PC2 y PC2↔PC3. Comprobar que los mensajes llegan correctamente y en el formato esperado.
Tolerancia a fallos	Simular la caída de PC3 y verificar que el sistema detecta el fallo, conmuta a la réplica y continúa operando sin interrupción.
Rendimiento	Medir el throughput (solicitudes/2 min) y la latencia (tiempo usuario→cambio semáforo) para los escenarios de la Tabla 1 del enunciado.

Consistencia de datos	Verificar que BD principal y réplica contienen los mismos registros tras un periodo de operación normal.
Estrés	Incrementar el número de sensores y la frecuencia de eventos para identificar el punto de saturación del sistema.

5.2 Casos de Prueba Funcionales

CP-01: Generación y recepción de eventos

- Precondición: PC1 y PC2 en ejecución.
- Acción: Iniciar los 3 procesos de sensores.
- Resultado esperado: PC2 recibe eventos de los 3 tipos dentro de los primeros 10 segundos. El Servicio de Analítica imprime los eventos recibidos.

CP-02: Detección de congestión

- Precondición: Sistema en ejecución completa.
- Acción: Configurar sensor de cámara para generar Lq=10 en INT_B2.
- Resultado esperado: Analítica imprime 'CONGESTIÓN detectada en INT_B2'. Semáforo INT_B2 cambia a verde extendido (30s).

CP-03: Priorización de vía (ambulancia)

- Precondición: Sistema en ejecución completa.
- Acción: Operador emite PRIORIZAR_VIA INT_A1 desde PC3.
- Resultado esperado: Semáforo INT_A1 cambia a verde por 60 segundos. Resto de semáforos en rojo. Analítica imprime la acción tomada.

CP-04: Consulta histórica

- Precondición: Sistema con al menos 2 minutos de operación.
- Acción: Emitir CONSULTA_HISTORICO INT_B2 con rango de tiempo válido.
- Resultado esperado: Monitoreo retorna lista de eventos registrados para INT_B2 en el periodo indicado.

CP-05: Failover ante caída de PC3

- Precondición: Sistema en ejecución completa con PC3 activo.
- Acción: Apagar PC3 (kill -9 al proceso o apagar la VM).
- Resultado esperado: Dentro de 6 segundos (3 timeouts × 2s), el sistema detecta el fallo. Las escrituras se redirigen a la réplica en PC2. El sistema continúa procesando eventos sin interrupciones visibles.

CP-06: Consistencia BD principal y réplica

- Precondición: Sistema en ejecución por 5 minutos.
- Acción: Pausar todos los sensores y consultar ambas bases de datos.
- Resultado esperado: BD principal y réplica contienen el mismo número de registros y los mismos datos (diferencia máxima de 1 registro por latencia de red).

5.3 Pruebas de Rendimiento (Tabla 1)

Las siguientes pruebas corresponden a los escenarios de la Tabla 1 del enunciado, comparando el diseño original (sin hilos) con el diseño multihilos en el BrokerZMQ:

Escenario	Configuración	Variable dependiente 1	Variable dependiente 2
E1-Original	1 sensor/tipo, cada 10s	Solicitudes almacenadas en BD en 2 minutos	Tiempo usuario→cambio semáforo (ms)
E2-Original	2 sensores/tipo, cada 5s	Solicitudes almacenadas en BD en 2 minutos	Tiempo usuario→cambio semáforo (ms)
E1-Multihilos	1 sensor/tipo, cada 10s	Solicitudes almacenadas en BD en 2 minutos	Tiempo usuario→cambio semáforo (ms)
E2-Multihilos	2 sensores/tipo, cada 5s	Solicitudes almacenadas en BD en 2 minutos	Tiempo usuario→cambio semáforo (ms)

Cada escenario se ejecutará 3 veces para obtener valores promedio y reducir la variabilidad. Se calculará media y desviación estándar de cada variable dependiente.

6. Estrategia de Obtención de Métricas

6.1 Métrica 1: Solicitudes almacenadas en BD (2 minutos)

Herramienta: Consulta SQL directa a la base de datos (SQLite) al finalizar el intervalo de 2 minutos.

- Procedimiento: Al iniciar cada experimento, se registra el timestamp T0. A los 2 minutos exactos (T0 + 120s), se ejecuta: SELECT COUNT(*) FROM eventos WHERE timestamp >= T0 AND timestamp <= T0+120s.
- Herramienta de medición: Python (sqlite3) + script automatizado que ejecuta la consulta y guarda el resultado en un archivo CSV.
- Unidad: número de registros insertados.

6.2 Métrica 2: Latencia usuario → cambio de semáforo

Herramienta: Timestamps Python con time.time() o time.perf_counter() de alta resolución.

- Procedimiento: El módulo de Monitoreo registra T1 = timestamp justo antes de enviar el comando REQ. El Servicio de Control de Semáforos registra T2 = timestamp justo después de aplicar el cambio. La latencia = T2 - T1 (ms). Ambos timestamps se guardan en la BD para análisis posterior.
- Herramienta de medición: Python time.perf_counter() (precisión de microsegundos). Los valores se exportan a CSV y se grafican con matplotlib.
- Unidad: milisegundos (ms).

6.3 Herramientas de Medición

Herramienta	Uso	Justificación
Python time.perf_counter()	Medición de latencias	Resolución sub-milisegundo, nativo en Python
SQLite + Python sqlite3	Conteo de registros en BD	Consultas SQL directas, sin overhead adicional
Script Python automático	Orquestación de experimentos	Garantiza repetibilidad y consistencia entre ejecuciones
matplotlib	Generación de gráficos	Visualización clara de resultados para el informe
CSV	Almacenamiento de resultados	Fácil procesamiento posterior y respaldo de datos

7. Implementación Inicial — PC1 y PC2

7.1 Descripción de la Implementación

La implementación inicial contempla todos los componentes de PC1 (sensores y broker) y PC2 (analítica y control de semáforos), junto con la escritura en la BD principal en PC3. El sistema es funcional en 3 computadores (físicos o virtuales) y puede demostrar los flujos completos de generación de eventos, detección de congestión y control de semáforos.

7.2 Tecnologías Utilizadas

- Lenguaje: Python 3.10+
- Mensajería: pyzmq (ZeroMQ para Python)
- Base de datos: SQLite 3 (módulo sqlite3 nativo de Python)
- Formato de mensajes: JSON (módulo json nativo)
- Configuración: Archivo config.json centralizado
- Control de tiempo: threading.Timer y time.sleep()

7.3 Estructura de Directorios

```
trafico_urbano/
  ├── config.json              # Parámetros globales del sistema
  ├── pc1/
  │   ├── sensor_camara.py     # Proceso sensor cámara
  │   ├── sensor_espira.py     # Proceso sensor espira inductiva
  │   ├── sensor_gps.py        # Proceso sensor GPS
  │   └── broker.py            # BrokerZMQ — agrega y reenvía eventos
  ├── pc2/
  │   ├── analitica.py          # Servicio de analítica y reglas
  │   ├── control_semaforos.py  # Servicio de control de semáforos
  │   ├── bd_replica.py         # Receptor PULL y escritor BD réplica
  │   └── health_check.py       # Monitor de disponibilidad de PC3
  └── pc3/
      ├── bd_principal.py       # Receptor PULL y escritor BD principal
      └── monitoreo.py          # Interfaz de monitoreo y consultas
```

8. Conclusiones

El diseño presentado en este informe refleja una arquitectura distribuida coherente con los principios estudiados en el curso. La separación de responsabilidades en tres nodos (sensores, procesamiento, persistencia) permite escalar o reemplazar componentes individualmente sin afectar al resto del sistema.

El uso de patrones ZeroMQ asíncronos (PUB/SUB, PUSH/PULL) garantiza que la pérdida de un componente no bloquee el sistema completo. El patrón REQ/REP para el módulo de monitoreo permite operaciones síncronas controladas cuando el usuario requiere respuesta inmediata.

La estrategia de tolerancia a fallos basada en Health Check y failover automático hacia la réplica asegura continuidad operacional ante la caída del nodo de persistencia principal, cumpliendo el requisito de transparencia para el cliente.

La cuadrícula 3x3 con 3 procesos de sensores (uno por tipo, cubriendo las 9 intersecciones) ofrece un equilibrio entre representatividad del sistema real y eficiencia computacional durante la ejecución y las pruebas.